Day 1 and Day 2
Student self study runbook

IT Service Desk Agent | Week 1 engineering foundations

A sequential guide for beginners using Windows or Mac. Includes setup, Python basics, curl demonstrations, every lesson prompt, test commands, failure clinics, reports, and Git submission.

One project throughout

Continue the same application, project folder, virtual environment, repository, and current Git branch. Run the prompts and commands on your own computer. Work in your own GitHub repository. Changes made by the instructor do not automatically appear in your local copy.

Where to begin

Your position	Start here
Day 1 completed	Use page 12 for the Day 2 restart. Refer to pages 3-6 if setup or curl materials are missing.
New student / fresh computer	Start on page 3. Complete setup and Day 1 before starting Day 2.
Before starting	Follow one lesson at a time and check the outcome before continuing.

What is grounded, and what the agent must inspect

The recorded Day 1 starter uses Python 3.11, FastAPI, pytest, an editable dev install, app.main:app, and port 8123. Recorded routes are /health, /employees/me, /services/vpn/status, and /tickets/{ticket_id}.

The repository itself was not supplied with this PDF. Authentication syntax, employee IDs, ownership mapping, and Day 1 test filenames must be read from each local project. Prompt D1-A creates exact local curl examples. The new Day 2 commands become available only after their corresponding prompts run.

Prepared 3 October 2026. Lesson prompts are consolidated execution prompts for this runbook. Test outcomes shown are expectations, not claims that this PDF executed your local repository.

Route through the two days

Pages	Sequence
3-4	Install/check tools; open project; create or activate .venv
5-6	D1-A: inspect the starter; create exact curl practice files
7	Business case and Python basics with D1-B
8-9	Start API; run all Day 1 curl demonstrations
10-11	Read/run tests; D1-C missing-ticket test; Day 1 checkpoint
12-13	Restart on Day 2; D2-A baseline inspection and test mapping
14-15	Fixtures and parameterisation: D2-B and D2-C
16-17	Ownership failure clinic: D2-D and execution
18-19	JSON Schema: D2-E and execution
20	20-case coverage: D2-F
21-22	HTML/JSON reports and final review: D2-G
23-24	Git review and submission; required student remote setup on page 24
25-26	Troubleshooting, quick commands, and completion checklist

Study sequence

Read the purpose. Paste one agent prompt. Inspect the changed code, then run the command. Compare your output with the expected result and record your explanation. Do not paste all prompts at once.

Where to type

Surface	Use
VS Code agent chat	Paste the labelled D1 / D2 prompts. Wait for completion and review its result.
VS Code terminal	Run shell commands from the project root. Use Windows PowerShell or macOS Terminal as labelled.
Editor / browser	Inspect source files, data, API documentation, and reports.

Suggested pacing: Day 1 about 2 hours after tools are ready; Day 2 about 3 hours including student practice. For a 2-hour Day 2 slot, complete remaining coverage cases independently. Resolve setup problems before proceeding to the next lesson.

1. Tools and project access

What / why

Install the tools that let us edit, execute, test, and save one Python application. Week 1 uses local synthetic data. No live model API key is required for these API lessons.

Tool	Required action
Python 3.11	Install Python 3.11 from python.org if absent. On Windows enable the launcher and PATH option. Follow your organisation's install policy.
VS Code	Install from code.visualstudio.com. Enable the Python extension.
Git + GitHub	Install Git from git-scm.com if absent. Sign into your own GitHub account.
Coding agent	Use the course-selected VS Code agent, signed in with the required account/subscription.
Starter repository	Instructor provides the working Day 1 repository. Do not create a second application if it already exists.

Check tools before creating anything

Windows PowerShell:

```
py -3.11 --version
git --version
curl.exe --version
```

macOS:

```
python3.11 --version
git --version
curl --version
```

Open the existing folder

Already have the Day 1 project? Open that folder and complete the remote check on page 24.
New students: clone once below, then complete page 24 before continuing with Python on page 4.

New students: open a parent folder where you keep projects in VS Code, then Terminal > New Terminal. Run these commands on Windows or Mac:

```
git clone https://github.com/Piyushjethwa3-ai/ai-qa-orchestration.git
cd ai-qa-orchestration
```

Then use File > Open Folder to open the newly created ai-qa-orchestration folder. Open a new terminal there and continue on page 4. Clone once per computer.

REQUIRED: Cloning sets the instructor URL as origin. Complete page 24 now to connect your own GitHub repository. Keep the instructor starter as upstream. Then return to page 4.

Everyone works on their own copy. We will build on this same folder each week.

2. Python environment and dependencies

What / why

A virtual environment keeps this project's Python packages together. Create it once. Activate it whenever you open a new terminal. If .venv already exists and works, skip creation.

Windows: create only if absent, then activate:

```
py -3.11 -m venv .venv
.\.venv\Scripts\Activate.ps1
```

Mac: create only if absent, then activate:

```
python3.11 -m venv .venv
source .venv/bin/activate
```

Check the selected environment - both systems

```
python --version
python -c "import sys; print(sys.executable)"
python -m pip --version
```

Expected: Python 3.11 and paths inside this project's .venv. In VS Code, open Command Palette > Python: Select Interpreter > select .venv.

Install the recorded starter dependencies

```
python -m pip install -e ".[dev]"
python -m pip show fastapi pytest httpx jsonschema uvicorn
```

Run this at the root containing pyproject.toml. The editable install links the local application to this environment; [dev] installs its declared development tools. If the project uses a different dependency file, Prompt D1-A must identify its correct command.

If PowerShell activation is blocked

Use the environment interpreter directly without changing system policy:

```
.\.venv\Scripts\python.exe -m pip install -e ".[dev]"
.\.venv\Scripts\python.exe -m pytest -v
```

For subsequent commands, replace the initial python with .\.venv\Scripts\python.exe. On Mac the equivalent is .venv/bin/python. Do not create a second venv just because activation failed.

> Installing into one Python and running another causes missing-package errors. Check which interpreter is active.

3. Inspect the actual starter

WHEN: after opening the project and activating .venv. Paste this into your own agent chat. It supplies the actual local details used on the next pages.

```
We are learning API testing from scratch in this existing IT Service Desk project.
Inspect before changing anything. Use the same folder, environment, repository and
current branch. Do not create another app or remove existing work. Do not commit or
push. Do not print secrets or read out .env values.

1. Identify the framework, entry point, dependency command, tests, API routes,
authentication mechanism and data paths. The recorded starter uses app.main:app on
port 8123 and pip install -e ".[dev]". Verify this locally.
2. Identify two synthetic employees A/B, a ticket owned by each, and a well-formed
missing ticket ID. Read ownership from prepared data, not from the response under
test. Document actual success/error contracts.
3. Create docs/day1-local-map.md with all file locations, the ownership table,
exact server/test commands, and whether pytest uses an in-process client or a live
server. Record Day 1 test file paths explicitly.
4. Add local practice files under .training/day1/. Generate curl config files named
01-health.curl, 02-profile.curl, 03-vpn.curl, 04-own-ticket.curl, 05-other-
ticket.curl, 06-missing-ticket.curl. Each must specify one localhost URL at port
8123 and the exact existing authentication needed for that scenario. Use Employee A
for profile/ticket scenarios. Health may omit authentication if public.
5. Use only synthetic local identity data. If credentials are required, reference
protected local credentials without displaying them. Exclude .training/ from Git.
Do not treat a training identity header as production authentication.
6. Also write docs/day1-curl-commands.md with the exact expanded curl commands for
PowerShell (curl.exe) and Mac (curl), explaining each header, route and expected
status. Verify the requests where possible; otherwise mark them unverified.
7. Record expected statuses from the documented starter: health/profile/VPN/own
ticket 200, other employee ticket 403, missing ticket 404. Flag any real contract
discrepancy rather than silently accepting it.
8. Run the existing tests in .venv. If a server is required, explain/start only the
local training server as appropriate. Report actual results and blockers.
Do not add Day 2 tests yet. Finish with the files I should open and commands I
should run next.
```

4. Read the project before running it

How

Open docs/day1-local-map.md. Confirm the real employee and ticket mapping. Open each data file at the location the agent reports. Do not assume the JSON files are at the repository root.

File / item	What the file contains
employees.json	Synthetic people and their permitted profile fields. Identify the employee ID.
tickets.json	Ticket records, status, and owner. Identify one ticket for each employee.
service_status.json	Prepared service state. This tells us whether VPN is healthy, degraded, or in outage.
app/main.py or actual entry point	Routes accept requests and return responses. Ownership must be enforced here or in called application logic.
Existing test file	Setup, request, expected result. The test is executable evidence.

Read JSON aloud

An object uses braces and named fields. A list uses square brackets and contains multiple items. A string is text in quotes. A number is not quoted. JSON uses true, false and null; Python uses True, False and None.

> This is our known starting data. We use it to decide what the application should return. We should not let the application tell us what the expected answer is.

Understand the curl practice files

The .curl files contain the real request URL and required training identity settings. The commands on pages 8-9 read those files. This avoids guessing an employee header or ticket owner. Open the files to inspect the request, but must not share credential values.

For plain expanded commands, open docs/day1-curl-commands.md. It is generated from the real application. Do not reuse another student's identity configuration without checking their data.

Checkpoint

Identify Employee A, Ticket A, Ticket B, and why A must not read B's ticket. If the starter is missing or not runnable, obtain the instructor's working starter before continuing.

5. Business case and Python basics

An employee reports a VPN problem after changing a password. Our service desk checks who they are, whether VPN has an outage, and any existing ticket. Today we test these building blocks. Later weeks add response evaluation, retrieval, safety, agent behaviour, orchestration and release evidence.

What / why

Learn variables, functions, conditions, loops, dictionaries, and JSON file reading using this same business story. You need enough Python to explain a test, not advanced programming on Day 1.

```
Extend this existing project with a beginner Python practice file:
practice/day1_python_basics.py
Use only synthetic data. Make it directly runnable from the project root.
Demonstrate in short labelled sections:
1. A variable holding an employee ID and a printed value.
2. A dictionary representing a ticket and reading one field.
3. A function accepting employee ID and ticket owner, returning a boolean.
4. if/else describing an allowed versus denied ownership decision.
5. A loop over a short ticket list.
6. Reading the actual tickets.json using json.load and a path based on
   __file__, handling the real JSON structure without changing the file.
Use plain comments and print statements; no network calls or writes.
Include beginner explanations of = versus ==, indentation, return, dictionary access and the
loop.
State clearly that this ownership function is a learning exercise, not
proof that the real API enforces permissions.
Run it and report the actual output. Preserve existing tests.
Do not commit or push.
```

```
python practice/day1_python_basics.py
```

Practise and inspect the result

Open the file. Read one section, predict its output, then run it. Change a sample owner so the local decision changes from allowed to denied. Keep the real JSON data unchanged.

A function groups reusable steps. A condition chooses what happens. A loop repeats work. An assertion, which we use next, checks whether the result matches our expectation.

6. Start the API and call it

Terminal A: run the application

Open a new terminal at the project root. Activate .venv using page 4. Use the recorded starter command below after D1-A confirms the entry point:

```
python -m uvicorn app.main:app --port 8123
```

Leave Terminal A running. Open http://127.0.0.1:8123/docs in the browser to inspect the FastAPI routes if documentation is enabled. Stop the server later with Ctrl+C.

Terminal B: send requests

Open another terminal at the same root. Curl calls a running server; it does not start the application. In Windows PowerShell use curl.exe to avoid the curl alias.

Windows - health and own profile:

```
curl.exe -i http://127.0.0.1:8123/health
curl.exe -i --config .training/day1/02-profile.curl
```

Mac - health and own profile:

```
curl -i http://127.0.0.1:8123/health
curl -i --config .training/day1/02-profile.curl
```

If D1-A finds health is protected, use 01-health.curl rather than the direct unauthenticated request. The recorded Day 1 health route was public.

Understand the command

Part	Meaning
curl / curl.exe	Send an HTTP request from the terminal.
-i	Include HTTP status and headers, followed by the body.
--config	Read URL/authentication settings from the generated local request file.
127.0.0.1:8123	Your own computer, listening on port 8123.
GET	The default method here: read information.

> The server terminal runs the application. The second terminal acts as its caller. The response includes a status code and a body. We need to inspect both.

7. Complete all Day 1 curl checks

PREREQUISITES: server running on port 8123; D1-A completed; execute from project root. Each config contains the real URL and identity settings. Run one command at a time and discuss the result.

Windows PowerShell

```
curl.exe -i --config .training/day1/03-vpn.curl
curl.exe -i --config .training/day1/04-own-ticket.curl
curl.exe -i --config .training/day1/05-other-ticket.curl
curl.exe -i --config .training/day1/06-missing-ticket.curl
```

macOS

```
curl -i --config .training/day1/03-vpn.curl
curl -i --config .training/day1/04-own-ticket.curl
curl -i --config .training/day1/05-other-ticket.curl
curl -i --config .training/day1/06-missing-ticket.curl
```

Scenario	Recorded Day 1 expectation
VPN: /services/vpn/status	200, with the prepared service state. Do not assume it is always healthy.
Own ticket	200, correct requested ticket and correct owner.
Other employee's ticket	403, access denied and no foreign ticket details.
Missing ticket	404, documented not-found response.

> 200 means this request succeeded. 403 means the caller is not allowed to access that ticket. 404 means the requested ticket was not found. A denied request is correct when the employee lacks permission.

Practice activity

For each request, write: caller, requested resource, expected status, actual status, and one body check. Ask: could a 200 response still be wrong? Yes, if it contains the wrong ticket or owner.

Curl does not normally treat HTTP 403/404 as a shell failure. Read the HTTP status displayed by -i. Pytest will later assert the intended outcome.

8. Read and execute Day 1 tests

What / why

Turn manual expectations into repeatable checks. The earlier starter was recorded with six passing tests; use the actual count reported by your current project, which may include subsequent additions.

How

Open the existing test file listed in docs/day1-local-map.md. Identify setup, the client request, and each assertion. An in-process test client may call the app without the uvicorn terminal; follow the local map.

```
python -m pytest --collect-only -q
python -m pytest -v
```

Command / concept	Meaning
--collect-only -q	List discovered tests without executing them.
-v	Show individual test names and outcomes.
Arrange	Prepare the caller and input.
Act	Make the API request.
Assert	Compare actual result with the requirement.

> We are writing the expected behaviour as code. A test can have several assertions: the status is correct, the ticket is correct, and the owner is correct. That is still one scenario.

Read a failure

Start with the test name. Identify the caller and ticket. Read the expected value and actual value. Check whether the cause is the application, the test, prepared data, or environment. Never change an expectation solely to get a green result.

Run a selected group

```
python -m pytest -v -k health
```

This selects test names matching health. Confirm selection in the output; a keyword is not a guarantee that a particular test exists.

Check your understanding

Choose one assertion and explain the risk it checks. Then identify what this test does not prove, such as ownership protection on another route.

9. Add a missing-ticket test

WHEN: after you understand one existing test. Add one case and inspect every changed line.

```
Add or review the missing-ticket regression test in this existing project.
Inspect existing tests before changing anything. If a suitable case already
exists, explain and use it rather than duplicating it.
Use the actual authenticated request mechanism and a well-formed ticket ID
verified absent from the prepared dataset. Expect the documented 404 error
response and no ticket details. Do not guess authentication headers.
Keep the test with the Day 1 tests. Add Arrange, Act, Assert comments.
Give the exact file path and exact single-test pytest command.
Run that case and the full normal suite. Preserve all existing checks.
Update docs/day1-local-map.md with the explicit Day 1 test paths and results.
Do not implement Day 2 topics. Do not commit or push.
```

Execute

Run the exact single-test command printed by the agent, then the full suite:

```
python -m pytest -v
```

> We supplied an ID that does not exist. The application should say not found. Our negative test passes when the application handles that situation correctly.

Day 1 checkpoint

You should now be able to explain the VPN story, open the JSON data, run basic Python, start the API, use curl, explain 200/403/404, and run or add a test.

Save the intended Day 1 changes using the review / stage / commit / push procedure on page 23. Use this commit message:

```
git commit -m "Complete Day 1 Python and API foundations"
git push -u origin main
```

Stage and review first (page 23). Verify origin is YOUR repository (page 24). Use your actual branch name if different from main. Do not push to upstream or the instructor repository.

Follow-up practice

Run the missing-ticket case again. Explain input, action, and assertion without reading the agent explanation. Do not add an AI model or UI as part of this assignment.

10. Restart the same project

What / why

Continue yesterday's application and confirm the starting point still works. Keep the current branch. Do not clone again or recreate a working .venv.

Windows PowerShell:

```
.\.venv\Scripts\Activate.ps1
```

Mac:

```
source .venv/bin/activate
```

If the local map says tests require a live API, start the server in a separate activated terminal BEFORE running pytest. In-process tests do not need it.

Both systems:

```
python --version
git status
git remote -v
git branch --show-current
python -m pytest --collect-only -q
python -m pytest -v
```

If tests require a live server, open another activated terminal and run:

```
python -m uvicorn app.main:app --port 8123
```

Understand the test commands

Command	Meaning
python -m pytest -v	All tests. Before adding Day 2 this is the earlier suite. Afterward it includes both days.
python -m pytest -v --ignore=tests/day2	Earlier tests, excluding the new Day 2 folder. If other tests exist, they are included too.
python -m pytest tests/day2 -v	Day 2 only, after the Day 2 test files have been created.

> Rerunning yesterday's tests tells us whether new work breaks earlier behaviour. We call that regression testing. Run these checks on your own computer.

Resolve baseline failures before continuing. Prompt D2-A records the exact Day 1-only file list so you can run that set precisely.

11. Map the Day 2 starting point

WHEN: beginning of Day 2. EVERY student runs this locally. The agent adapts to the actual project without inventing IDs, headers or test paths.

```
We are starting Day 2 of the beginner IT Service Desk API testing course.
Continue this existing Day 1 application in the same folder, virtual
environment, repository, and current branch. Do not create another app,
repository, branch or environment. Preserve existing behaviour and tests.
Do not commit, push, print secrets, or expose .env contents.

Inspect the actual application, tests, dependencies, sample data,
identity mechanism and startup procedure. Use docs/day1-local-map.md
if present, but verify it against the current code.

Before creating Day 2 tests:
1. Record explicit Day 1 test paths and the exact command that runs
   only those paths in docs/day2-instructor-map.md. Do not move them.
2. Verify the Day 1 baseline in the correct environment. Report actual
   results; investigate errors without weakening tests.
3. Record actual employee IDs, ticket ownership, routes, fields and
   documented status-code expectations.
4. Record server startup commands for Windows and Mac if tests need
   a server. State clearly if tests use an in-process client instead.
5. Use tests/day2/ for new Day 2 files. Do not create empty files yet.
6. Plan pytest option --day2-fault with default 'none', to be added by
   the ownership lesson. Register options in root conftest.py or an
   early-loaded plugin, not only in a nested tests/day2/conftest.py.
   This keeps full-suite fault commands discoverable.
7. Finish with the exact baseline command, the existing test to open
   for recap, and any remaining blocker.
Do not implement the rest of Day 2 yet.
```

Review the generated reference

Open docs/day2-instructor-map.md and the suggested recap test. Identify its input, request and assertion. Save the explicit Day 1 command for later reruns.

> The agent helps prepare code. Review what changed and decide whether the expectations match the business rule.

12. Reusable fixtures

WHAT: functions that supply test setup or data. WHY: avoid copying the same client and employee setup into every test.

> A data fixture is prepared sample data. A pytest fixture is a function that supplies something a test needs. The fixture prepares; the test checks.

```
Implement only the Day 2 fixtures lesson in this existing project.
Reuse existing test clients, identity helpers and fixtures.
Add tests/day2/conftest.py only for additional shared setup needed here.
Provide beginner-readable fixtures for Employee A, Employee B and their
known owned tickets using actual prepared data. Do not derive expected
ownership from the API response under test.
Create tests/day2/test_01_fixtures.py with two real API cases:
- A retrieves A's ticket.
- B retrieves B's ticket.
Check success status, requested ticket ID and expected owner.
Use clear Arrange, Act, Assert comments. Avoid unnecessary autouse or
advanced fixture scopes. Preserve existing Day 1 tests and behaviour.
Run this file and the recorded Day 1-only suite.
Update docs/day2-instructor-map.md with actual results, file locations,
and a beginner explanation of how fixture names are supplied to tests.
Do not implement later lessons or commit/push.
```

Inspect the code

Open tests/day2/conftest.py. Identify @pytest.fixture, the function name, and its return value or yield. Open test_01_fixtures.py and identify the fixture requested by name in the test arguments.

```
python -m pytest tests/day2/test_01_fixtures.py -v
```

EXPECTED: two own-ticket cases pass on the correct baseline. Read actual output instead of promising a fixed total for the whole suite.

Independent practice

Run the prompt in your agent, inspect the files and execute the command. Identify which fixture supplies the caller and which assertion checks the owner.

> Changing one reusable setup function can help many tests. That is also why we review fixtures carefully: an incorrect fixture can affect many results.

13. Multiple input cases

WHAT: parameterisation executes the same logic with different input rows. WHY: cover different outcomes without copying entire functions.

> Each row is a scenario. The employee, ticket and expected result can change while the request steps remain the same.

```
Create tests/day2/test_02_parameterized.py using existing fixtures.
Add one readable parameterised API test with descriptive case IDs:
1. Employee A requests A's ticket.
2. Employee B requests B's ticket.
3. Employee A requests B's ticket.
4. Employee B requests A's ticket.
5. A valid employee requests a verified well-formed missing ticket ID.
Use the real identity mechanism and documented response contracts.
For success, check requested ticket and owner. For denial, verify the
error contract and absence of protected foreign-ticket data. For the
missing ticket, verify the documented not-found response.
Do not accept arbitrary alternative codes to make a test pass.
Include explanations of @pytest.mark.parametrize, input names, rows and case IDs.
Run the file and update docs/day2-instructor-map.md with actual results.
Label scenarios repeated from fixture teaching: they do not add unique
business coverage. Preserve previous tests. Do not commit or push.
```

```
python -m pytest tests/day2/test_02_parameterized.py -v
```

Inspect

Inspect the decorator, input names, data rows and function. Find the separate scenario names in the output. One test function can produce several independently reported cases.

Expected result

Five parameterised scenarios pass on the correct baseline. The two cross-employee requests pass their tests when the API denies access as required.

> A negative test passes when invalid or forbidden input is handled correctly. An HTTP error response does not automatically mean our test should fail.

Independent practice

Choose one row. Explain caller, requested ticket, expected status and expected body. Before adding a sixth row, explain what new risk it covers.

14. Prepare the ownership failure clinic

WHAT: test the API boundary for employee isolation. WHY: another person's ticket must remain private even when its ID is known. This tests BR02.

```
Create tests/day2/test_03_ownership.py with two-direction cross-employee
access checks, documented denial responses, and checks that protected
foreign-ticket values are not exposed.

Add a TEST-ONLY pytest option --day2-fault with default 'none' and initial
value 'ownership'. Register it in the root conftest.py or an early-loaded
plugin; preserve existing configuration. Reject unknown values.
With --day2-fault=ownership, bypass only ownership enforcement in an
isolated test application instance or equivalent test-only setup.

Constraints:
- Exercise the actual application endpoint/request path.
- Do not manufacture responses in the test or change assertions.
- Do not disable authentication as a substitute for ownership.
- Do not change persistent data or normal application behaviour.
- Never enable the fault by default. Restore overrides after the run.
- If the existing live-server architecture cannot isolate this fault,
  report the blocker. Do not introduce a persistent unsafe toggle.

Verify these three runs in order:
1. Normal ownership tests PASS.
2. The same tests FAIL because foreign-ticket access is allowed when
   --day2-fault=ownership is supplied.
3. Normal tests PASS again with the option omitted.

Do not use xfail or skip to hide the deliberate failures.
Update the generated reference map with enforcement code location, fault setup,
exact commands, actual outcomes and plain-English explanation.
Do not commit or push.
```

Check the fault setup

Verify this setup before running the fault demonstration. A setup crash is not proof the ownership test detects the defect. Verify that failure messages show the incorrect access response or disclosure.

> You will introduce one controlled defect and keep the expectation unchanged. If the test catches the defect, it provides evidence that this check is useful.

15. Run the ownership clinic

1. Correct baseline

```
python -m pytest tests/day2/test_03_ownership.py -v
```

EXPECTED: access to the other employee's ticket is denied, and the tests pass.

> The application blocked access. That is the behaviour our test requires.

2. Deliberately faulty behaviour

```
python -m pytest tests/day2/test_03_ownership.py -v --day2-fault=ownership
```

EXPECTED: the relevant ownership checks fail and pytest exits nonzero. Read the assertion: expected denial, actual access or leaked data. Other unrelated tests need not fail.

> This run bypassed ownership enforcement only in the test application. The unchanged tests detected access that should have been denied. We do not change the expected result to make this green.

3. Normal behaviour again

```
python -m pytest tests/day2/test_03_ownership.py -v
```

EXPECTED: tests pass again. The fault flag is absent; cleanup prevents it carrying over.

What to record

Evidence	Your explanation
Rule	Employees can read only their own tickets.
Input	Employee A requesting Employee B's ticket.
Expected	Denied, without protected ticket details.
Actual under fault	Access allowed or protected data disclosed.
Verification	Same tests pass with normal enforcement restored.

This is a controlled defect-detection exercise. If you discover a real application defect instead, fix application logic, preserve the regression check, and explain that separate change.

16. Prepare JSON Schema validation

WHAT: rules for response fields and types. WHY: valid JSON can still omit required information or return unusable values.

```
Implement reusable JSON Schema checks based on this project's actual
API contract. Use jsonschema and update the existing dependency file.
Create tests/day2/test_04_schema.py for actual employee, ticket,
service-status and relevant error responses. Check the expected HTTP
status before applying the corresponding success/error schema.
Define required fields and types explicitly. Do not infer mandatory
fields solely from one observed response. Document contract decisions.
Keep schema files and shared validator helpers beginner-readable.

Extend the root/early-loaded --day2-fault option with missing-status
and wrong-status-type. Inject these defects into an isolated test
application's ticket response generation path, not into the response
inside a test. Do not alter persistent data or normal application code
behaviour. Restore test overrides after the run.

Verify baseline passes, each injected fault is detected, and baseline
passes afterward. If framework response validation catches malformed
data first, preserve it and explain the actual failure. Do not disable
validation to manufacture a schema failure. If isolation is unsupported,
report the blocker rather than leaving a persistent fault toggle.

Add a separately labelled standalone validator demonstration if needed
to show a missing field and wrong type directly. Do not count this as
an API case. Explain required, properties, object and string.
Update the generated reference map with paths, commands and actual results.
Preserve earlier tests. Do not commit or push.
```

Inspect the code

Open the generated ticket schema. The required list makes fields mandatory. Properties describes their rules. A property is not mandatory just because it is listed under properties.

> Schema checks structure. Ownership checks permission. We need both because someone else's ticket can have a perfectly valid structure.

17. Run schema checks and faults

1. Normal responses

```
python -m pytest tests/day2/test_04_schema.py -v
```

EXPECTED: schema and HTTP contract checks pass on the normal baseline.

2. Missing status

```
python -m pytest tests/day2/test_04_schema.py -v --day2-fault=missing-status
```

EXPECTED: at least the affected ticket contract test fails. The error may identify a missing required field, a server-side response validation failure, or an unexpected HTTP error.

3. Wrong status type

```
python -m pytest tests/day2/test_04_schema.py -v --day2-fault=wrong-status-type
```

EXPECTED: the affected check detects the malformed response. Explain what actually happened, including whether the framework rejected it first.

4. Restore normal run

```
python -m pytest tests/day2/test_04_schema.py -v
```

> The malformed response violates our agreed contract. The test should detect that. If the application catches it first, that is still a contract failure for this injected run, but it is not evidence that JSON Schema was the component that caught it.

Practice exercise

Predict outcomes for: missing required field; a number where text is required; correctly structured foreign ticket. Then run the demonstrations and explain which check catches each situation.

Regression check

```
python -m pytest -v --ignore=tests/day2
```

This excludes the Day 2 folder. Use the explicit Day 1 paths from the generated reference map if other earlier tests are also present.

18. Reach meaningful Week 1 coverage

The target is at least 20 distinct API scenarios across both days. It is not 20 new cases today and not a count of individual assertions.

```
Review all Day 1 and Day 2 tests. Create docs/week01-api-case-matrix.md
with case ID, requirement/contract, scenario, employee context, expected
result, category, executable pytest node ID, and overlap notes.
Ensure at least 20 meaningful distinct API scenarios across Week 1:
- Valid profiles and own-ticket requests.
- Cross-employee access denial in both directions.
- Missing/unknown identities according to the real contract.
- Missing and malformed ticket IDs with documented outcomes.
- Controlled healthy/degraded/outage service-status cases.
- Successful and error response schema contracts.
Add missing cases to tests/day2/test_05_week1_coverage.py.
Use supported endpoints and isolated prepared data. Do not invent routes,
weaken expectations or change requirements merely to reach the count.
Document unsupported coverage precisely.
Do not count duplicate teaching scenarios, assertions, standalone schema
exercises or fault-injected reruns as new baseline API scenarios.
Run the full normal suite. Report unique scenarios separately from total
pytest executions. Preserve earlier tests. Update the generated reference map.
Do not commit or push.
```

Run in sequence

```
python -m pytest --collect-only -q
python -m pytest -v --ignore=tests/day2
python -m pytest tests/day2 -v
python -m pytest -v
```

> Collection shows what pytest found. Execution shows what passed. The case matrix shows which risks we covered. We need all three views.

Review the matrix before accepting the 20-case claim. A large collected count may include duplicate demonstrations or standalone validator checks.

Independent practice

Each student chooses one negative case and explains how it differs from the other scenarios. Record unsupported cases and complete remaining supported cases as follow-up, not by silently declaring coverage complete.

19. Generate the baseline reports

What / why

HTML is readable in a browser. JSON is structured output for tools. Save reports so reviewers can inspect the test run, not just hear that it passed.

```
python -m pip install pytest-html pytest-json-report
python -c "from pathlib import Path; Path('reports/day2').mkdir(parents=True, exist_ok=True)"
```

Windows PowerShell: full suite

The backtick at the end of each continued line must be the last character, with no trailing spaces.

```
python -m pytest -v `
  --html=reports/day2/baseline.html --self-contained-html `
  --json-report --json-report-file=reports/day2/baseline.json
```

Mac: full suite

```
python -m pytest -v \
  --html=reports/day2/baseline.html --self-contained-html \
  --json-report --json-report-file=reports/day2/baseline.json
```

Open the HTML

Windows:

```
Start-Process .\reports\day2\baseline.html
```

Mac:

```
open reports/day2/baseline.html
```

Open baseline.json in the VS Code editor. Inspect case names, outcomes, duration and any failure details. The files are created only after the report command runs.

> This baseline report covers both days. Results belong to a particular code and dataset version. A report alone does not prove we covered every important risk.

20. Preserve fault and restored evidence

Windows PowerShell

```
python -m pytest tests/day2/test_03_ownership.py -v `
  --day2-fault=ownership `
  --html=reports/day2/ownership-fault.html --self-contained-html `
  --json-report --json-report-file=reports/day2/ownership-fault.json

python -m pytest tests/day2/test_03_ownership.py -v `
  --html=reports/day2/ownership-restored.html --self-contained-html `
  --json-report --json-report-file=reports/day2/ownership-restored.json
```

Mac

```
python -m pytest tests/day2/test_03_ownership.py -v \
  --day2-fault=ownership \
  --html=reports/day2/ownership-fault.html --self-contained-html \
  --json-report --json-report-file=reports/day2/ownership-fault.json

python -m pytest tests/day2/test_03_ownership.py -v \
  --html=reports/day2/ownership-restored.html --self-contained-html \
  --json-report --json-report-file=reports/day2/ownership-restored.json
```

Run each command separately. The injected-fault run should fail; the restored run should pass. Do not overwrite the failure report with the passing run.

Agent prompt D2-G: final review

```
Prepare the Day 2 submission without staging, committing or pushing.
Update the existing dependency file and README for Windows/Mac setup,
server needs, explicit Day 1 commands, Day 2 commands, fault demonstrations
and reports. Create docs/day2-run-evidence.md with actual run time, tested
commit plus uncommitted state, dataset identity, counts and limitations.
Use N/A for model/prompt usage in these deterministic Week 1 API tests.
Inspect reports for credentials or unnecessary sensitive environment data;
keep only sanitised teaching evidence. Exclude .env, .venv, .training and
caches from Git. Verify fault modes are isolated and off by default.
Run the full normal suite, report real results and list changed files.
Do not invent results, change branches or change Git remotes.
```

21. Review commit and push

Before the first submission, complete page 24. Keep the same project folder and current branch. The instructor starter is **upstream**; your own GitHub repository is **origin**.

1. Check the branch and destination

```
git branch --show-current
git remote -v
git status
git diff --stat
git diff
```

Confirm both origin fetch and push URLs use YOUR GitHub account. Upstream should point to Piyushjethwa3-ai/ai-qa-orchestration. If origin still points to the instructor, stop and complete page 24. Git diff shows tracked edits; inspect untracked files separately.

2. Stage only the intended files

In VS Code Source Control, review each intended file and click + to stage it. Do not stage .env, virtual environments, .training, caches or unrelated work. For a selected file, the terminal alternative is:

```
git add README.md
git diff --cached --stat
git diff --cached
```

3. Commit locally

For Day 1, use the message on page 11. For Day 2:

```
git commit -m "Complete Day 2 fixtures schemas and API reports"
```

If Git reports nothing to commit, check whether changes were saved, staged or already committed. Do not create an artificial change.

4. Push explicitly to your own origin

If your current branch is main, run the following. Otherwise replace main with the branch name shown in step 1. Do not create or rename a branch for this lesson.

```
git push -u origin main
git status
git log -1 --oneline
```

The -u option sets your own origin branch as the tracking destination. Open YOUR repository and that branch on GitHub to confirm the commit appears. Authenticate through the normal GitHub process. If push is rejected, inspect the cause; do not force-push.

Commit saves our reviewed changes locally. Push uploads them to your own portfolio repository. Each student owns their tests and evidence.

22. Connect your own GitHub repository

Complete this page immediately after cloning on page 3, or now if you already have a local clone. **Clone once. Do not run git init, clone again or delete your work.** These commands work on Windows and Mac from the project root.

1. Create an empty destination on GitHub

Sign into your own account. Create a repository named ai-qa-orchestration. Leave README, .gitignore and licence unselected because the local clone already contains project files. Copy its HTTPS URL. Access to the instructor repository does not give you permission to push there.

2. Inspect before changing remotes

```
git remote -v
git branch --show-current
```

If origin already points to your own repository, skip step 3. If origin points to the instructor and no upstream exists, run step 3. If upstream already exists or the arrangement differs, inspect its URL before proceeding; do not overwrite a valid destination blindly.

3. Preserve the starter and set your destination

Replace YOUR_USERNAME with your actual GitHub username. Do not paste the placeholder unchanged.

```
git remote rename origin upstream
git remote add origin https://github.com/YOUR_USERNAME/ai-qa-orchestration.git
git remote -v
```

Expected: origin fetch/push URLs use YOUR account. Upstream fetch/push URLs use Piyushjethwa3-ai. Renaming a remote does not copy or delete your local files. Upstream is a reference for instructor updates; origin is your upload destination.

4. Check your commit identity

```
git config user.name
git config user.email
```

If absent, substitute your name and preferred Git commit email. This config applies only to this repository:

```
git config user.name "YOUR NAME"
git config user.email "YOUR GIT COMMIT EMAIL"
```

5. Continue the lesson and make the first push

Return to page 4 for Python setup. When ready to submit, review and commit using page 23. For a current branch named main:

```
git push -u origin main
```

Use your actual branch name if different. If no local changes exist, you can still push the cloned starter history. The -u option replaces any inherited tracking of upstream/main with your own origin/main. Later bare git push commands use that tracking destination.

Already completed Day 1? Keep your changes. Follow steps 1-4, then submit on page 23. Share your own repository URL and current branch with the instructor.

23. Troubleshoot without guessing

Symptom	First check / action
Python command not found	Windows: py -3.11 --version. Mac: python3.11 --version. Install/repair Python before creating .venv.
Package not found	Check python -m pip --version and sys.executable. Activate the project .venv, then install dependencies there.
Editable install fails	Confirm project root contains pyproject.toml with the dev extra. Ask the agent to inspect the actual dependency setup.
PowerShell blocks activation	Use .\.venv\Scripts\python.exe directly. See page 4.
Connection refused in curl	Check Terminal A is running uvicorn on 8123 and request URL matches it.
Port 8123 already in use	Reuse your existing training server or stop it with Ctrl+C in its terminal. Do not kill unrelated processes.
Unexpected 403 on own ticket	Check caller identity and ownership mapping in local prepared data.
Expected 403, got 200	Confirm identity and requirement, then investigate ownership enforcement.
fixture not found	Check fixture spelling, conftest.py scope, imports and test location.
--day2-fault unrecognised	D2-D must run first. Register option in root conftest.py or an early-loaded plugin.
Report flags unrecognised	Install pytest-html and pytest-json-report in the active interpreter.
No tests collected	Check working directory, test file paths and test_ function names.
Fault run crashes during setup	This is not evidence of defect detection. Inspect setup and ensure the request reached the application.
Report has sensitive values	Remove unnecessary metadata/captured values before sharing; rerun sanitised evidence.
Nothing to commit	Check whether edits were saved/staged or already committed. Do not manufacture a change.

Investigate each failure in order

Test name > caller/input > expected > actual > evidence > cause > correction > rerun. Never weaken an assertion merely to obtain a pass.

24. Quick reference and completion

Every session

Activate .venv using your OS instructions, then run from project root:

```
python --version
python -m pytest -v
git status
git remote -v
```

API and test commands

```
python -m uvicorn app.main:app --port 8123
python practice/day1_python_basics.py
python -m pytest --collect-only -q
python -m pytest -v --ignore=tests/day2
python -m pytest tests/day2 -v
python -m pytest -v
```

Do not run uvicorn in the test terminal if it blocks you entering the next command. Use Terminal A for the server and Terminal B for requests/tests. The practice file and Day 2 files exist only after their prompts run.

Prompt timing

Prompt	When
D1-A / D1-B / D1-C	Starter inspection / Python practice / missing-ticket regression
D2-A / D2-B / D2-C	Day 2 baseline / fixtures / parameterisation
D2-D / D2-E / D2-F / D2-G	Ownership / schemas / coverage / final review

Completion evidence

Each student submits the repository/current branch, setup instructions, at least 20 distinct Week 1 API cases, fixture and schema helpers, sanitised HTML/JSON reports, and one explained fault detection followed by a normal passing run.

Optional Week 1 tag: first run git tag --list week-01. Only if it does not exist, create and push the checkpoint after committing verified work:

```
git tag -a week-01 -m "Verified Week 1 API checkpoint"
git push origin week-01
```

Sources: supplied IT Service Desk Agent Training Case Study, Week 1 and BR01/BR02; supplied Student Guide, Week 1; recorded Day 1/Day 2 teaching flow. Technical references: docs.pytest.org, python-jsonschema.readthedocs.io, github.com/pytest-dev/pytest-html, github.com/numirias/pytest-json-report.